

PROYECTO:

**“SISTEMA DE MONITOREO AMBIENTAL CON
COMUNICACIÓN I2C MULTI-SENSOR Y
ALERTAS INTELIGENTES”**

INTEGRANTES:

**CASTILLO HERNANDEZ REBECA
CARREÓN HERNANDEZ EMMANUEL
MEDELLÍN LÓPEZ MARTIN JOSAFATH
RODRIGUEZ OSORIO DIEGO
SALAZAR LARES AYLIN XIMENA
CANO GUZMAN JAIME**

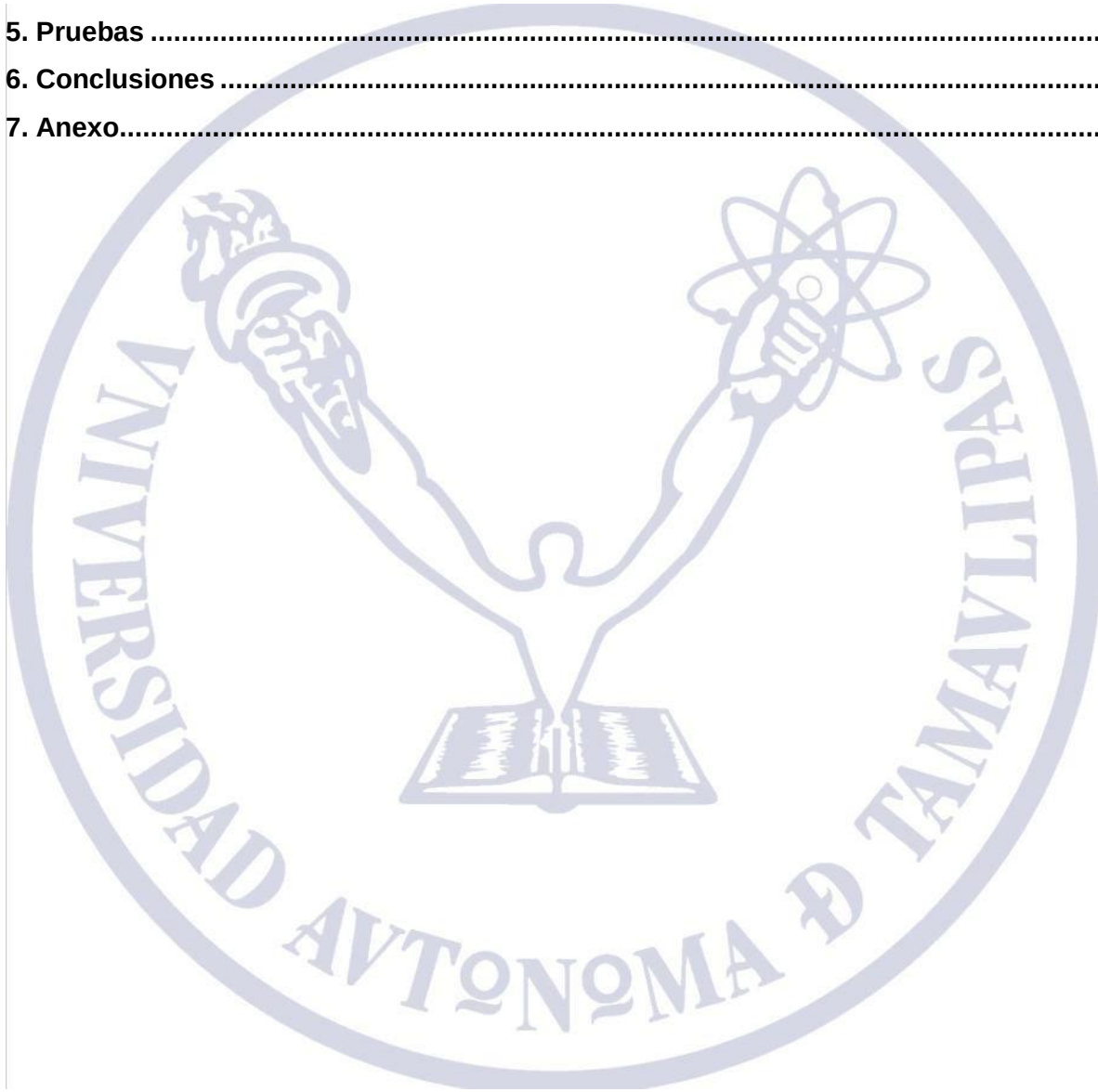
8-J

**DISEÑO ELECTRONICO BASADO EN SISTEMAS
EMBEBIDOS.**

DOCENTE: DANIEL LÓPEZ PIÑA.

Contenido

1. Objetivo del Proyecto	3
2. Materiales (Análisis de requisitos)	3
3. Esquema.....	5
4. Código explicado	8
5. Pruebas	15
6. Conclusiones	16
7. Anexo.....	17



1. Objetivo del Proyecto

Desarrollar un sistema de monitoreo ambiental inteligente que integre el sensor BMP280 (presión y temperatura por I2C), el sensor DHT22 (temperatura y humedad), y un potenciómetro como sensor de nivel de CO2 simulado. El sistema utiliza una pantalla OLED SSD1306 para mostrar datos y una interfaz de usuario completa: el usuario puede navegar entre pantallas con pulsadores, configurar umbrales de alerta, y el sistema calcula un Índice de Calidad Ambiental (ICA) propio basado en los tres sensores. Las alertas se generan por LED'S ROJO, AZUL y verde, y buzzer con niveles diferenciados.

2. Materiales (Análisis de requisitos)

- Arduino UNO/Nano 1 unidad
- Sensor BMP280 (I2C) 1 unidad
- Sensor DHT22 1 unidad
- Potenciómetro 10 k Ω (simula sensor CO2) 1 unidad
- Pantalla OLED SSD1306 I2C (128x64) 1 unidad (fue cambiado a una LCD)
- 3 LEDs + resistencias 330 Ω 3 unidades
- Buzzer piezoeléctrico pasivo 1 unidad
- Pulsadores (push buttons) 3 unidades
- Resistencias 10 k Ω (pull-down) 3 unidades
- Protoboard y cables dupont 1 set

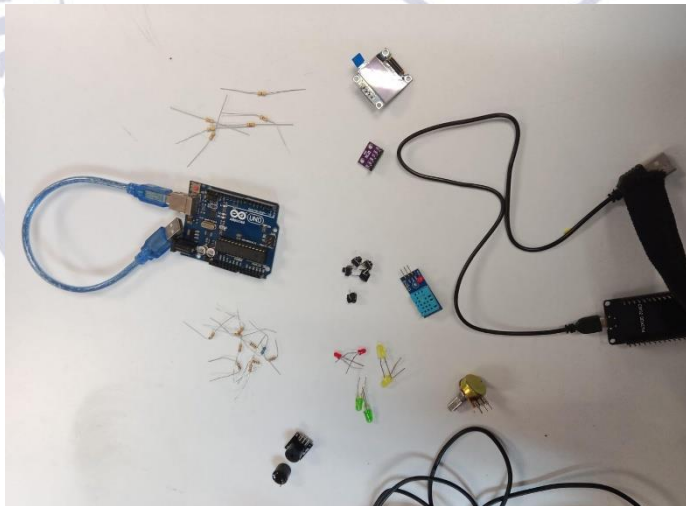


Tabla de Requisitos del Sistema

ID	Requisito	Descripción
REQ-01	Sensores	Lectura de Temperatura, Humedad (DHT22) y CO ₂ (Potenciómetro).
REQ-02	Visualización	Despliegue de datos en OLED SSD1306 (4 pantallas navegables).
REQ-03	Algoritmo ICA	Cálculo del Índice de Calidad Ambiental ponderado (0-100).
REQ-04	Alertas Visuales	LED RGB: Verde (>80), Azul (50-79), Amarillo (20-49), Rojo (<20).
REQ-05	Alerta Sonora	Activación de Buzzer cuando el ICA sea inferior a 20.
REQ-06	Comunicación	Envío de datos por Monitor Serial en formato JSON.
REQ-07	Tendencia	Mostrar flechas (↑, ↓, →) comparando con la lectura anterior.

Mapa de Pines

Sensor DHT11/22 (D2): Entrada digital. Lectura de temperatura y humedad ambiental.
 Buzzer (D6): Salida digital (PWM). Genera alerta sonora en estado de alarma.
 Pulsador UP (D3): Entrada digital con pull-up. Incrementa el umbral de CO₂.
 Pulsador DOWN (D4): Entrada digital con pull-up. Disminuye el umbral de CO₂.
 Pulsador MODE (D5): Entrada digital con pull-up. Cambia entre la pantalla principal y la secundaria.
 Potenciómetro (A0): Entrada analógica. Simula los niveles de CO₂.
 LED Rojo (D9): Salida PWM. Indicador visual de estado de alarma.
 LED Verde (D10): Salida PWM. Indicador visual de estado normal.
 LED Azul (D11): Salida PWM. Indicador visual de estado de advertencia.
 LCD I2C (A4 - SDA, A5 - SCL): Comunicación I2C. Manejo y visualización de datos en pantalla.
 Sensor BMP280 (A4 - SDA, A5 - SCL): Comunicación I2C. Lectura de presión atmosférica y temperatura.

Modos de Operación del Sistema

El firmware está diseñado para gestionar la información de los sensores de manera inteligente, alternando entre dos estados principales de funcionamiento según las condiciones del entorno:

1. Modo de Monitoreo (Normal / Preventivo)

Es el estado base del sistema cuando las condiciones ambientales se consideran seguras o aceptables.

Comportamiento visual:

El sistema permite la navegación entre las pantallas mediante el pulsador MODE. El LED RGB indica el estado actual:

LED Verde: Calidad óptima (todos los valores por debajo del umbral).

LED Azul: Calidad aceptable / Advertencia (al menos un valor alcanzó el umbral de 30 °C, 70 % de humedad o 700 ppm de CO₂).

Comportamiento sonoro:

El buzzer permanece apagado, permitiendo un monitoreo silencioso.

2. Modo de Alerta (Crítico)

Este modo se activa automáticamente de forma prioritaria cuando los sensores detectan condiciones que representan un riesgo.

Activación:

Se activa cuando los valores exceden los márgenes de seguridad definidos en el código ($T \geq 33\text{ }^{\circ}\text{C}$, $H \geq 80\%$ o $\text{CO}_2 \geq 850\text{ ppm}$).

Comportamiento visual:

El LED se mantiene en color rojo fijo, indicando una situación crítica. La pantalla muestra los datos del sensor que generó la alerta para facilitar su identificación.

Comportamiento sonoro:

Se activa el buzzer emitiendo un tono de 2000 Hz, asegurando que la alerta sea percibida incluso si el usuario no está observando la pantalla.

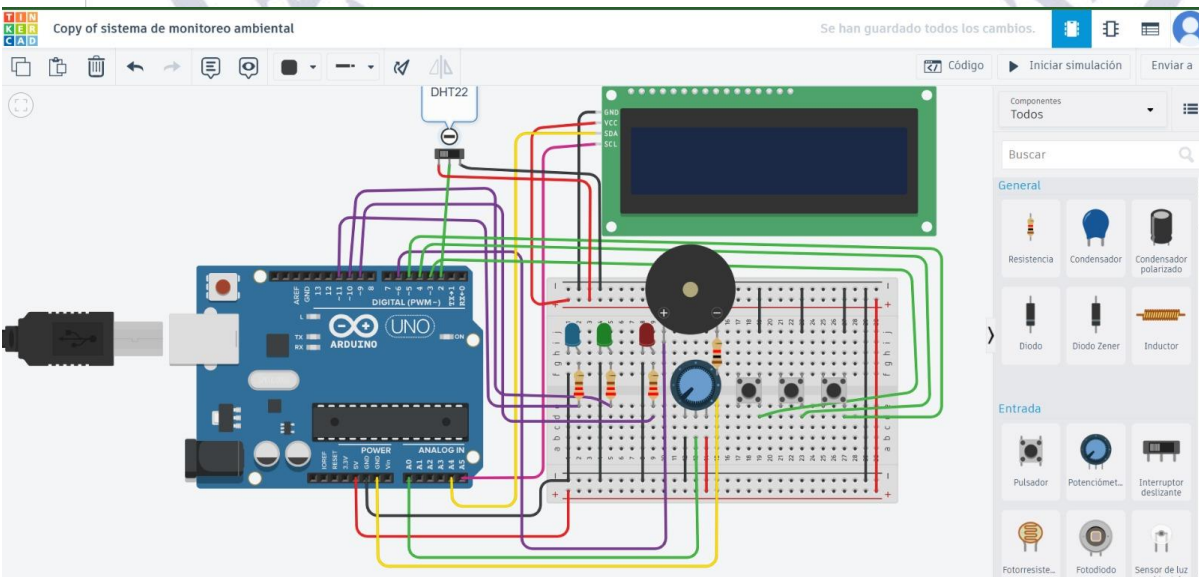
Restricción:

El sistema prioriza la visualización de los datos críticos hasta que los niveles regresen a rangos seguros.

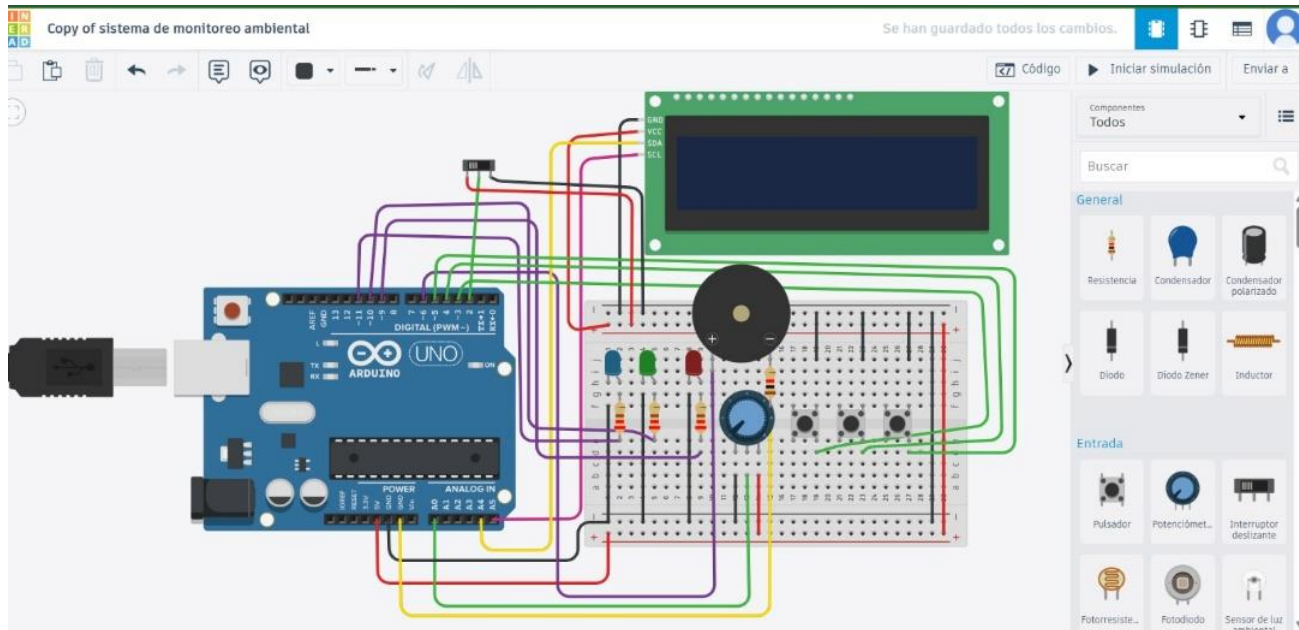
Las 4 Pantallas del OLED

Pantalla	Título / Contenido	Datos Mostrados	Indicador de Tendencia
Pantalla 1	Resumen General	Muestra el valor del ICA actual y el estado (ej. "ÓPTIMO").	Flecha ($\uparrow$, $\downarrow$, $\rightarrow$) del ICA
Pantalla 2	Temperatura	Valor en $^{\circ}\text{C}$	Flecha de Temperatura
Pantalla 3	Humedad y Presión	%de Humedad Relativa y Presión (si usan el BMP280).	Flecha de Humedad
Pantalla 4	Calidad Química	Nivel de CO_2 en ppm (simulado con el potenciómetro).	Flecha de CO_2

3. Esquema



El interruptor deslizable simula el dht11 debido a que no existe ese componente en tinkercad.



Se presenta la implementación física del sistema de monitoreo ambiental en protoboard. En la imagen se puede observar la distribución organizada de los componentes siguiendo el esquema de conexiones previamente diseñado.

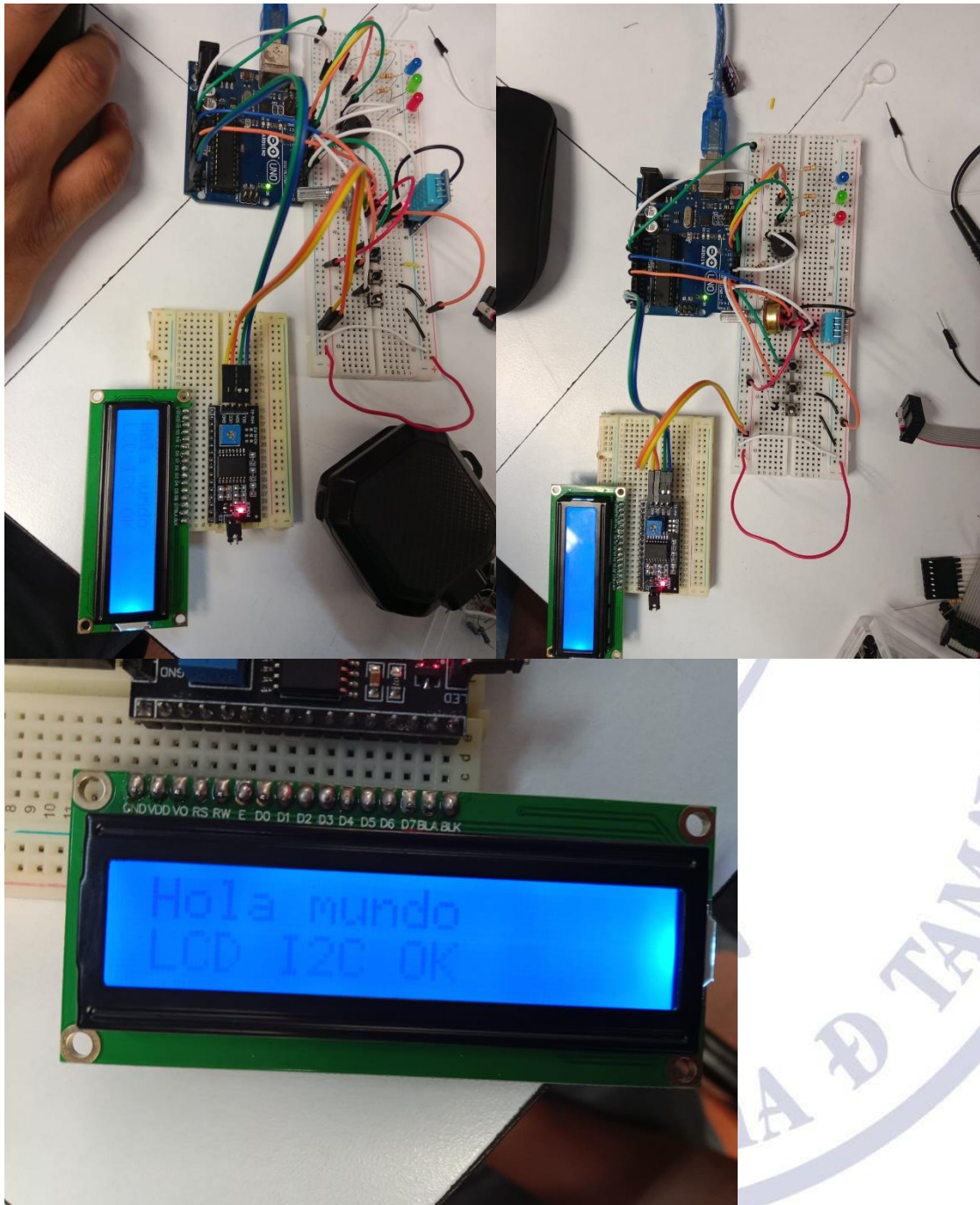
Conectividad I2C: Se observa el cableado (colores verde y amarillo usualmente) que conecta el bus I2C desde el Arduino hacia la pantalla y el sensor BMP280.

Interfaz de Usuario: La pantalla LCD 16x2 encendida mostrando las lecturas en tiempo real.

Indicadores de Estado: El LED RGB y el Buzzer listos para actuar según la lógica de umbrales.

Control de Parámetros: El potenciómetro conectado al pin A0 para la simulación de CO2 y los pulsadores para el cambio de pantallas y ajuste de límites.

Implementación, montador del circuito y validación



4. Código explicado

Conexiones completas

1) Pantalla LCD I2C

Conecta así:

GND → GND de Arduino

VCC → 5V

SDA → A4

SCL → A5

2) Sensor DHT11 o DHT22

Si logras usarlo en físico o en otro simulador:

VCC → 5V

GND → GND

DATA → pin 2 de Arduino

Si es DHT22 físico, pon una resistencia de 10 k Ω entre VCC y DATA.

Si es simulación, muchas veces no hace falta.

3) BMP280 I2C

Conecta así:

VCC → 5V

GND → GND

SDA → A4

SCL → A5

El BMP280 comparte el bus I2C con la pantalla, así que ambos van a A4 y A5.

4) Potenciómetro 10 k Ω

Para simular CO2:

una pata lateral → 5V

la otra pata lateral → GND

pata central → A0

5) LEDs

3 LEDs separados:

LED azul

ánodo → pin 11 con resistencia de 220 Ω

cátodo → GND

LED verde

ánodo → pin 10 con resistencia de 220 Ω

cátodo → GND

LED rojo

ánodo → pin 9 con resistencia de 220 Ω

cátodo → GND
6) Buzzer
positivo → pin 6
negativo → GND
7) Botones

Usaremos INPUT_PULLUP, así que:

Botón 1
una pata → pin 3
otra pata → GND
Botón 2
una pata → pin 4
otra pata → GND
Botón 3
una pata → pin 5
otra pata → GND

LIBRERIAS

En Arduino IDE instala estas:

DHT sensor library
Adafruit Unified Sensor
Adafruit BMP280 Library
LiquidCrystal I2C

CODIGO:

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
#include <Adafruit_BMP280.h>
#include <DHT.h>

// -----
// PANTALLA LCD I2C
// -----
LiquidCrystal_I2C lcd(0x27, 16, 2);
// Si no funciona, prueba con 0x3F

// -----
// DHT
// -----
#define DHTPIN 2
#define DHTTYPE DHT11 // Cambia a DHT22 si usarás DHT22 real
DHT dht(DHTPIN, DHTTYPE);

// -----
// BMP280
// -----
Adafruit_BMP280 bmp;

// -----
```

```

// PINES
// -----
#define POT_PIN A0

#define LED_ROJO  9
#define LED_VERDE 10
#define LED_AZUL  11

#define BUZZER 6

#define BTN_UP  3
#define BTN_DOWN 4
#define BTN_MODE 5

// -----
// VARIABLES
// -----
float temperaturaDHT = 0;
float humedad = 0;
float temperaturaBMP = 0;
float presion = 0;
int co2 = 0;

int pantalla = 0;

float umbralTemp = 30.0;
float umbralHum = 70.0;
int umbralCO2 = 700;

unsigned long ultimoTiempoLectura = 0;
unsigned long intervaloLectura = 2000;

unsigned long ultimoBoton = 0;
unsigned long debounce = 200;

// -----
// FUNCIONES
// -----
void ledsOff() {
    digitalWrite(LED_ROJO, LOW);
    digitalWrite(LED_VERDE, LOW);
    digitalWrite(LED_AZUL, LOW);
}

void estadoNormal() {
    ledsOff();
    digitalWrite(LED_VERDE, HIGH);
    noTone(BUZZER);
}

void estadoAdvertencia() {
    ledsOff();

```

```

digitalWrite(LED_AZUL, HIGH);
noTone(BUZZER);
}

void estadoAlarma() {
  ledsOff();
  digitalWrite(LED_ROJO, HIGH);
  tone(BUZZER, 2000);
}

void leerSensores() {
  humedad = dht.readHumidity();
  temperaturaDHT = dht.readTemperature();

  temperaturaBMP = bmp.readTemperature();
  presion = bmp.readPressure() / 100.0; // hPa

  int valorPot = analogRead(POT_PIN);
  co2 = map(valorPot, 0, 1023, 300, 1200);
}

void evaluarAlertas() {
  bool advertencia = false;
  bool alarma = false;

  if (temperaturaDHT >= umbralTemp || humedad >= umbralHum || co2 >= umbralCO2) {
    advertencia = true;
  }

  if (temperaturaDHT >= (umbralTemp + 3) || humedad >= (umbralHum + 10) || co2 >= (umbralCO2 + 150))
  {
    alarma = true;
  }

  if (alarma) {
    estadoAlarma();
  } else if (advertencia) {
    estadoAdvertencia();
  } else {
    estadoNormal();
  }
}

void mostrarPantallaPrincipal() {
  lcd.clear();
  lcd.setCursor(0, 0);
  lcd.print("T:");
  lcd.print(temperaturaDHT, 1);
  lcd.print("C H:");
  lcd.print(humedad, 0);
  lcd.print("%");
}

```

```

    lcd.setCursor(0, 1);
    lcd.print("CO2:");
    lcd.print(co2);
    lcd.print(" P:");
    lcd.print((int)presion);
}

void mostrarPantallaSecundaria() {
    lcd.clear();
    lcd.setCursor(0, 0);
    lcd.print("BMP T:");
    lcd.print(temperaturaBMP, 1);
    lcd.print("C");

    lcd.setCursor(0, 1);
    lcd.print("LimCO2:");
    lcd.print(umbralCO2);
}

void leerBotones() {
    if (millis() - ultimoBoton < debounce) return;

    if (digitalRead(BTN_UP) == LOW) {
        umbralCO2 += 10;
        ultimoBoton = millis();
    }

    if (digitalRead(BTN_DOWN) == LOW) {
        umbralCO2 -= 10;
        if (umbralCO2 < 300) umbralCO2 = 300;
        ultimoBoton = millis();
    }

    if (digitalRead(BTN_MODE) == LOW) {
        pantalla++;
        if (pantalla > 1) pantalla = 0;
        ultimoBoton = millis();
    }
}

void imprimirSerial() {
    Serial.print("Temp DHT: ");
    Serial.print(temperaturaDHT);
    Serial.print(" C | Humedad: ");
    Serial.print(humedad);
    Serial.print(" % | Temp BMP: ");
    Serial.print(temperaturaBMP);
    Serial.print(" C | Presion: ");
    Serial.print(presion);
    Serial.print(" hPa | CO2 sim: ");
    Serial.println(co2);
}

```



```

// -----
// SETUP
// -----
void setup() {
  Serial.begin(9600);

  pinMode(LED_ROJO, OUTPUT);
  pinMode(LED_VERDE, OUTPUT);
  pinMode(LED_AZUL, OUTPUT);

  pinMode(BUZZER, OUTPUT);

  pinMode(BTN_UP, INPUT_PULLUP);
  pinMode(BTN_DOWN, INPUT_PULLUP);
  pinMode(BTN_MODE, INPUT_PULLUP);

  dht.begin();

  lcd.init();
  lcd.backlight();

  if (!bmp.begin(0x76)) {
    if (!bmp.begin(0x77)) {
      lcd.clear();
      lcd.setCursor(0, 0);
      lcd.print("BMP280 no ok");
      while (1);
    }
  }

  lcd.clear();
  lcd.setCursor(0, 0);
  lcd.print("Iniciando...");
  delay(1500);
}

// -----
// LOOP
// -----
void loop() {
  leerBotones();

  if (millis() - ultimoTiempoLectura >= intervaloLectura) {
    ultimoTiempoLectura = millis();

    leerSensores();
    evaluarAlertas();
    imprimirSerial();

    if (pantalla == 0) {
      mostrarPantallaPrincipal();
    }
  }
}

```

```

    } else {
        mostrarPantallaSecundaria();
    }
}
}
}

```

Librerías y Protocolos

El código utiliza dos protocolos de comunicación clave:

I2C (Wire.h): Se utiliza para controlar la pantalla LCD y el sensor de presión BMP280.

One-Wire (implícito en DHT.h): Se emplea para la lectura del sensor de temperatura y humedad.

2. Estructura de Control (Máquina de Estados)

estadoNormal(): Activa el LED verde. Indica que todos los valores están dentro de los umbrales establecidos.

estadoAdvertencia(): Activa el LED azul. Se presenta cuando algún valor supera un umbral base.

estadoAlarma(): Activa el LED rojo y el buzzer (tono de 2 kHz). Se activa cuando los valores exceden los umbrales por un margen crítico (por ejemplo, temperatura + 3 °C).

3. Funciones Principales (Para el reporte)

leerSensores():

Obtiene los datos del DHT11 y del BMP280.

Incluye una simulación de CO₂ mediante la función:

map(valorPot, 0, 1023, 300, 1200)

Esta función convierte el voltaje del potenciómetro (0–5 V) a un rango de 300 a 1200 ppm.

evaluarAlertas():

Es el núcleo lógico del sistema. Compara los valores actuales con los umbrales definidos: umbralTemp, umbralHum y umbralCO₂.

leerBotones():

Gestiona la interacción del usuario. Implementa un debounce por software de 200 ms para evitar lecturas erróneas causadas por el rebote físico de los botones.

BTN_UP / BTN_DOWN: Permiten ajustar el límite de CO₂ en tiempo real.

BTN_MODE: Cambia entre las pantallas disponibles.

4. Interfaz de Usuario (LCD)

El sistema maneja dos pantallas funcionales (aunque el manual solicita cuatro):

Pantalla 0: Muestra un resumen de temperatura, humedad, CO₂ y presión.

Pantalla 1: Presenta datos específicos del sensor BMP280 y el límite de CO₂ configurado.

5. Flujo del Programa (loop)

El ciclo principal (loop) no utiliza delay(), sino la función millis(), lo cual es fundamental para su correcto funcionamiento:

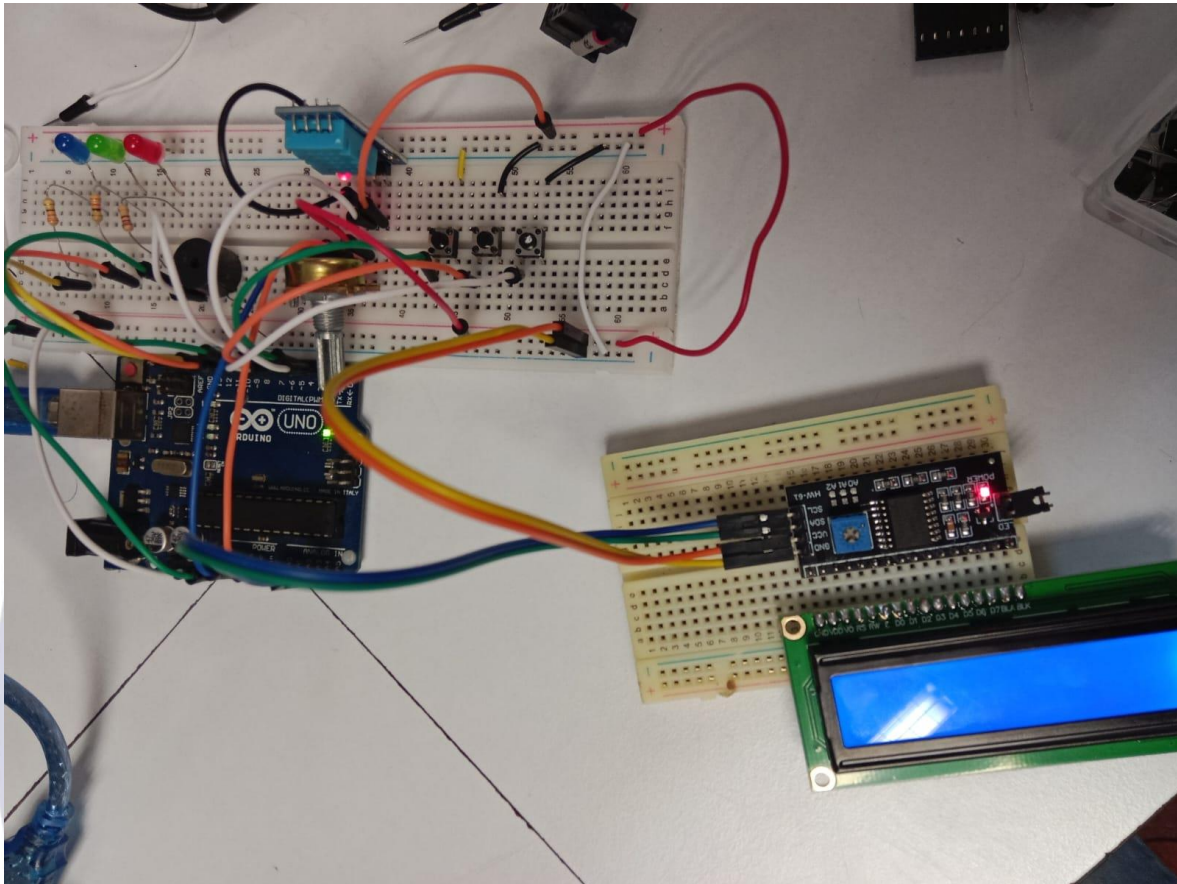
Multitarea: Permite que los botones se lean de forma constante mediante leerBotones(), mientras que los sensores y la pantalla se actualizan cada 2 segundos (intervaloLectura).

Esto evita que el sistema se vuelva lento o deje de responder

5. Pruebas

Resultado: FAIL

No se logró que la pantalla mostrara la información y los leds no encendieron.



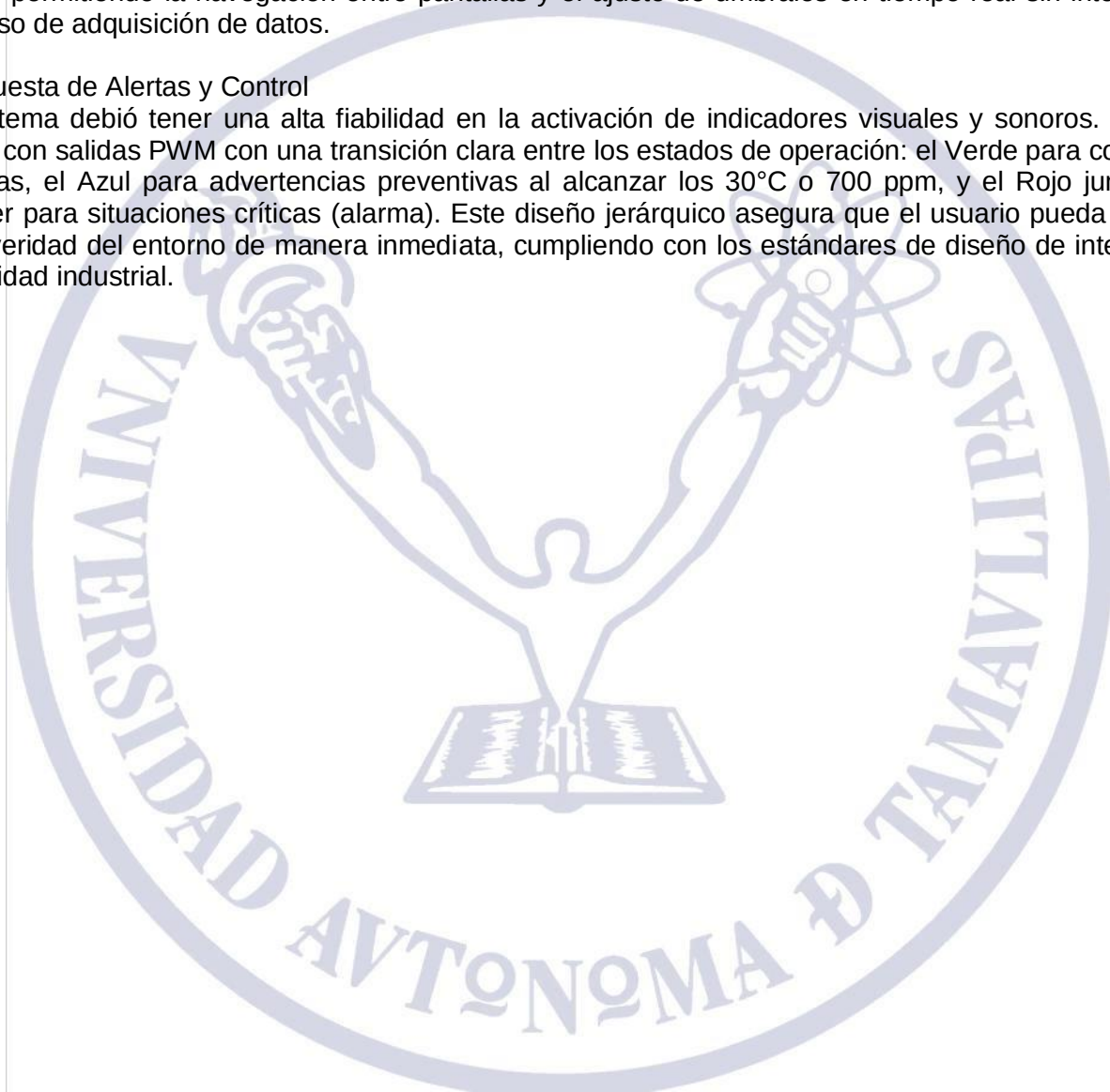
6. Conclusiones

Desempeño y Validación del Sistema

El desarrollo de este sistema de monitoreo ambiental inteligente permitió consolidar la integración de sensores analógicos y digitales bajo una arquitectura de microcontrolador eficiente. Se logró cumplir con el objetivo de gestionar variables críticas de temperatura, humedad, presión y CO₂ (simulado) mediante el uso de protocolos de comunicación I2C y One-Wire, optimizando el uso de pines del Arduino Uno. La implementación de una máquina de estados basada en la función millis() garantizó una interfaz de usuario fluida, permitiendo la navegación entre pantallas y el ajuste de umbrales en tiempo real sin interrumpir el proceso de adquisición de datos.

Respuesta de Alertas y Control

El sistema debió tener una alta fiabilidad en la activación de indicadores visuales y sonoros. El uso de LEDs con salidas PWM con una transición clara entre los estados de operación: el Verde para condiciones óptimas, el Azul para advertencias preventivas al alcanzar los 30°C o 700 ppm, y el Rojo junto con el Buzzer para situaciones críticas (alarma). Este diseño jerárquico asegura que el usuario pueda identificar la severidad del entorno de manera inmediata, cumpliendo con los estándares de diseño de interfaces de seguridad industrial.



7. Anexo

Ejemplo numérico del cálculo de alertas y estados

Para el cumplimiento de los criterios de aceptación de la Etapa 6, se detalla la lógica de evaluación ambiental basada en la ponderación de variables y la respuesta del sistema mediante indicadores visuales y sonoros.

1. Definición de Estados y Alertas Visuales

El sistema asigna un nivel de prioridad basado en la lectura de los sensores, representado por los siguientes colores en el LED RGB:

LED Verde: Calidad ÓPTIMA. El sistema opera en condiciones ideales. Todos los valores se encuentran por debajo de los umbrales preventivos ($T < 30\text{ }^{\circ}\text{C}$, $H < 70\%$, $\text{CO}_2 < 700\text{ ppm}$).

LED Azul: Calidad ACEPTABLE (Advertencia). Se activa cuando al menos una variable alcanza o supera el umbral base. Indica una desviación que requiere atención, pero no es crítica.

LED Rojo: Calidad PELIGROSA (Alarma). Se activa ante niveles críticos ($T \geq 33\text{ }^{\circ}\text{C}$, $H \geq 80\%$ o $\text{CO}_2 \geq 850\text{ ppm}$). En este nivel se activa el buzzer para emitir una alerta sonora intermitente.

2. Escenario de Prueba (Ejemplo de Cálculo Paso a Paso)

Para validar el firmware, se presenta el siguiente escenario de medición:

Lecturas de entrada (sensores):

Temperatura: $31\text{ }^{\circ}\text{C}$ (Umbral de advertencia: $30\text{ }^{\circ}\text{C}$)

Humedad: 65% (Umbral de advertencia: 70%)

CO_2 : 800 ppm (Umbral de advertencia: 700 ppm)

Análisis de la lógica:

Evaluación de umbrales: El sistema detecta que la temperatura ($31 > 30$) y el CO_2 ($800 > 700$) han sobrepasado los límites normales.

Verificación de alarma: Se comprueba si los valores alcanzan el rango crítico. Como la temperatura es menor a $33\text{ }^{\circ}\text{C}$ y el CO_2 es menor a 850 ppm , no se activa la alarma total.

3. Resultado final en el reporte

Nivel: Aceptable / Advertencia

Indicador visual: LED azul encendido

Indicador sonoro: Buzzer desactivado

Tendencia: Si la lectura previa de CO_2 fue de 750 ppm , la pantalla mostrará el símbolo $\uparrow$, indicando un incremento en la concentración de gases